

Department Library Management System

Role-based book discovery, cart requests, checkout, returns, feedback, and administration

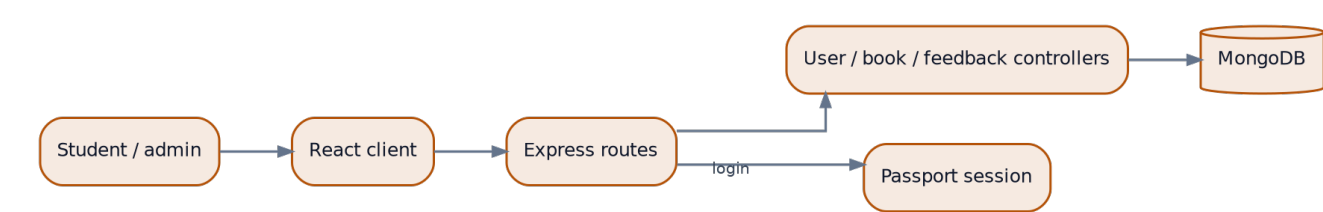
Project context: Biosciences and Bioengineering Department, IIT Roorkee	Status: Implemented MERN application
---	--------------------------------------

A departmental library application with student and administrator experiences. Users search books, add selections to a cart, track borrowed items, request returns, edit profiles, and submit feedback. Administrators manage books, checkout requests, returns, users, and query statuses.

Technology stack

React	Node.js / Express	MongoDB / Mongoose	Passport
bcrypt	REST APIs	Role-based UI	Bulk book import

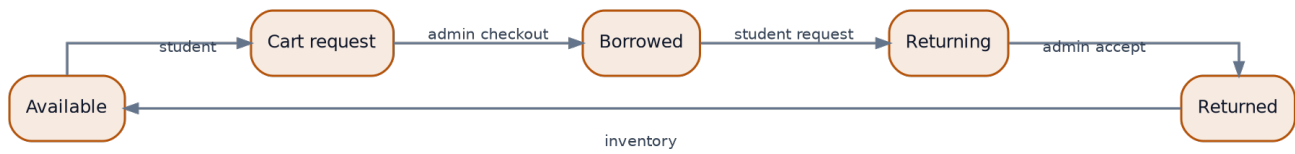
System at a glance



Book discovery Searches title, author, genre, and publisher, with inventory views.	Borrow workflow Moves selections from a user cart into a borrowed list after approval.
Return workflow Tracks returning status and lets administrators complete returns.	Feedback workflow Users submit queries while administrators accept, reject, or close them.

System design and component ownership

The React client calls a single Express route layer. Controllers own user, book, and feedback behavior, while Mongoose models store users, books, borrowed/cart arrays, and feedback records. Passport sessions provide login state.



Component	Responsibility
React client	Login, profile, books, cart, borrowing, feedback, and admin pages.
Express routes	Defines user, book, cart, return, search, filter, and feedback endpoints.
User controller	Registration, bcrypt hashing, profile updates, Passport login, and logout.
Book controller	Bulk import, search, cart, checkout, borrowed lists, and returns.
Feedback controller	Creates queries and manages accepted/rejected/submitted states.
MongoDB models	Stores users, books, recent activity, and feedback.

Key design decisions

- Separate user, book, and feedback controllers behind one route module.
- Use a cart as an approval staging area before checkout.
- Represent return requests with an explicit returning status.
- Provide different client workflows for students and administrators.

Core execution flow

A student searches and places a book in the cart. An administrator checks it out, which removes it from the cart and adds it to the borrowed list. The student later marks it returning, and the administrator completes the return.



Implementation notes

- Registration hashes passwords with bcrypt and generates a unique user ID.
- Book search uses case-insensitive regular expressions over several fields.
- Bulk import uses MongoDB bulkWrite with ISBN-based upserts.
- Checkout validates the user, cart entry, and inventory record before updating the borrowed array.
- Feedback endpoints support create, delete, accept, reject, and per-user views.

Representative API surface

```
router.post("/register", userController.registerUser);
router.post("/login", userController.loginUser);
router.get("/allBook", bookController.getAllBooks);
router.get("/search/:id", bookController.searchBooks);
router.post("/addToCart", bookController.addToCart);
router.post("/checkout", bookController.checkout);
router.post("/returnStatusBook", bookController.returnStatusBook);
router.post("/returnBooks", bookController.returnBooks);
```

Route names and controller ownership are taken directly from the inspected repository.

Engineering review and production path

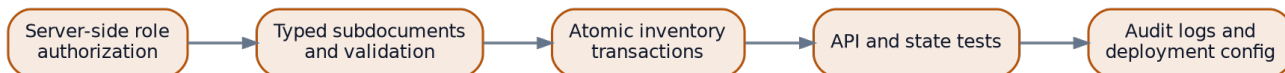
Engineering strengths

- Covers a complete real-world approval workflow, not only CRUD screens.
- Clear separation of user, book, and feedback domains.
- Bulk ISBN upsert is useful for importing an existing catalog.
- Student and administrator roles make the product behavior easy to demonstrate.

Current limitations

- Several endpoints need server-side role and authorization middleware.
- Borrowed and cart fields are untyped arrays in the current Mongoose model.
- Some inventory-count updates are commented out in the controller.
- Validation, HTTP status consistency, tests, and transaction safety need strengthening.

Recommended next steps



Why this is relevant to AI training work

- Offers state-machine tasks with cart, borrowed, returning, and returned states.
- Supports evaluation of authorization, inventory, duplicate, and race-condition edge cases.
- Demonstrates the ability to inspect code and identify both functionality and defects.
- Provides a concrete system where generated tests can verify business rules.

Repository:

<https://github.com/SidheshwarSarangal/department-vikram-project>

Scope and provenance:

Prepared from the repository route layer, user and book controllers, Mongoose models, setup guide, and functional page descriptions.

